Applied Type Theory

System F: полиморфизм второго порядка

Воронов Михаил Сергеевич

ВМК МГУ · Осень 2026

План лекции

1 **Квантор ∀ и правила System F**

2 Стиль Карри и вывод типов Hindley–Milner

3 Параметричность и свойства программ

Тождественные функции для разных типов

$\lambda x : \mathsf{Int}.\, x \quad : \mathsf{Int} \to \mathsf{Int}$

$\lambda x : \mathsf{Bool}.\, x \quad : \mathsf{Bool} \to \mathsf{Bool}$

В $\lambda_{\to}$ тождественные функции для Int и Bool записываются с разными аннотациями параметра.

В System F тип становится **параметром функции**, поэтому обе записи обобщаются одним определением.

$\mathsf{I} := \forall \alpha.\, \alpha \to \alpha$

В примерах к ядру System F добавлены базовые типы Int, Bool и произведения, их кодировки приведены в дополнительной части.

Полиморфная тождественная функция

$$\mathsf{id} \coloneqq \Lambda\alpha.\,\lambda x : \alpha.\, x \quad : \forall\alpha.\,\alpha \to \alpha$$

Абстракция $\Lambda\alpha$ связывает **типовую переменную**, а λx — **термовую**.

$$\mathsf{id}[\mathsf{Int}]\,42 : \mathsf{Int} \qquad \mathsf{id}[\mathsf{Bool}]\,\mathsf{true} : \mathsf{Bool}$$

$$\mathsf{id}[\mathsf{Int} \to \mathsf{Int}] : (\mathsf{Int} \to \mathsf{Int}) \to (\mathsf{Int} \to \mathsf{Int})$$

В $M[\tau]$ передаётся тип, а в $M\,N$ — терм.

Типы System F

$$\sigma, \tau ::= \alpha \mid \sigma \to \tau \mid \forall \alpha. \tau$$

Квантор $\forall$ допускается **в любой части типа**, включая тип аргумента функции.

$$\forall \alpha. \alpha \to \alpha \qquad (\forall \alpha. \alpha \to \alpha) \to \mathsf{Int}$$

Стрелки группируются вправо, а область действия квантора продолжается до конца типа или закрывающей скобки.

$$\forall \alpha. \alpha \to \alpha \;=\; \forall \alpha. (\alpha \to \alpha)$$

Второй порядок здесь означает квантификацию по типам, тогда как функции высших порядков были уже в $\lambda_{\to}$.

Термы в стиле Чёрча

$$M, N ::= x \mid \lambda x : \sigma . M \mid M\,N \mid \Lambda\alpha . M \mid M[\tau]$$

Два вида параметров

$$\lambda x : \sigma . M \quad \longleftrightarrow \quad M\,N$$

$$\Lambda\alpha . M \quad \longleftrightarrow \quad M[\tau]$$

В абстракции $\Lambda\alpha . M$ переменная α обозначает тип, а тело M является термом.

Свободные переменные и подстановка типов

$$\mathrm{FTV}(\alpha) = \{\alpha\} \qquad \mathrm{FTV}(\sigma \to \tau) = \mathrm{FTV}(\sigma) \cup \mathrm{FTV}(\tau)$$

$$\mathrm{FTV}(\forall \alpha.\, \tau) = \mathrm{FTV}(\tau) \setminus \{\alpha\}$$

$$\mathrm{FTV}(\forall \beta.\, \alpha \to \beta) = \{\alpha\}$$

Свободные переменные подставляемого типа должны остаться свободными.

$$(\forall \beta.\, \alpha \to \beta)[\alpha := \beta] \;=\; \forall \gamma.\, \beta \to \gamma$$

Переименовываем связанную β в свежую γ, чтобы подставленная β осталась свободной.

Контекст System F

$$\Gamma ::= \varnothing \mid \Gamma, \alpha : * \mid \Gamma, x : \tau$$

Контекст содержит объявления типовых переменных $\alpha : *$ и термовых переменных $x : \tau$.

$$\Gamma = \alpha : *, \beta : *, f : \alpha \to \beta, x : \alpha$$

$$\Gamma \vdash f\ x : \beta$$

Каждое имя объявляется **до использования** и выбирается свежим среди имён своего вида, поэтому $x : \alpha$ может стоять только после $\alpha : *$.

Формирование типов

$$\frac{\alpha : * \in \Gamma}{\Gamma \vdash \alpha : *} \qquad \frac{\Gamma \vdash \sigma : * \qquad \Gamma \vdash \tau : *}{\Gamma \vdash \sigma \to \tau : *}$$

$$\frac{\Gamma, \alpha : * \vdash \tau : *}{\Gamma \vdash \forall \alpha. \tau : *}$$

При расширении корректного контекста выбираем свежую переменную.

$$\varnothing \vdash \forall \alpha. \alpha : *$$

Тип $\forall \alpha. \alpha$ корректно сформирован и является **пустым типом** чистой System F.

Правила для переменной и стрелки

$$\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma}\ \text{Var}$$

$$\frac{\Gamma, x : \sigma \vdash M : \tau}{\Gamma \vdash \lambda x : \sigma . M : \sigma \to \tau} \to I$$

$$\frac{\Gamma \vdash M : \sigma \to \tau \qquad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \to E$$

Правила для стрелки те же, что в $\lambda_{\to}$, только типы σ и τ теперь могут содержать кванторы.

Введение полиморфизма

$$\frac{\Gamma, \alpha : * \vdash M : \tau}{\Gamma \vdash \Lambda\alpha.M : \forall\alpha.\tau} \quad \forall I$$

Выводим $M : \tau$ при **произвольном типе** α, после чего связываем α абстракцией Λ.

$\alpha \notin \mathrm{dom}_{\mathrm{type}}(\Gamma)$

Типовая переменная α должна быть свежей для внешнего контекста Γ.

Условие свежести при обобщении

$\Gamma = \alpha : *, x : \alpha \qquad \Gamma \vdash x : \alpha$

Можно ли присвоить x тип $\forall \beta.\, \beta$?

$\Gamma \nvdash x : \forall \beta.\, \beta$

При обобщении по свежей β тип x остаётся α, заданным внешним контекстом.

$\lambda x : \alpha.\, \Lambda \beta.\, x : \alpha \to \forall \beta.\, \alpha$

Устранение квантора: инстанцирование

$$\frac{\Gamma \vdash M : \forall \alpha . \tau \qquad \Gamma \vdash \sigma : *}{\Gamma \vdash M[\sigma] : \tau[\alpha := \sigma]} \; \forall E$$

$\mathsf{id} : \forall \alpha . \alpha \to \alpha$

$\mathsf{id}[\mathsf{Bool}] : \mathsf{Bool} \to \mathsf{Bool}$

В теле τ заменяем **свободные вхождения** α выбранным типом σ.

Правило допускает подстановку полиморфного типа σ, содержащего кванторы: такое инстанцирование называется **импредикативным**.

Вывод типа тождественной функции

$$\alpha : *, x : \alpha \vdash x : \alpha$$

$$\frac{\alpha : *, x : \alpha \vdash x : \alpha}{\alpha : * \vdash \lambda x : \alpha.\, x : \alpha \to \alpha} \to I$$

$$\frac{\alpha : * \vdash \lambda x : \alpha.\, x : \alpha \to \alpha}{\varnothing \vdash \Lambda\alpha.\, \lambda x : \alpha.\, x : \forall\alpha.\, \alpha \to \alpha} \forall I$$

Сначала правило $\to I$ связывает x, затем $\forall I$ связывает α.

Две β-редукции

Подстановка терма

$$(\lambda x : \sigma. M)\, N \to_{\beta} M[x := N]$$

Подстановка типа

$$(\Lambda \alpha. M)[\tau] \to_{\beta_\forall} M[\alpha := \tau]$$

Во втором правиле заменяем α типом τ в аннотациях и типовых аргументах терма M.

При подстановке связанные переменные переименовываются по мере необходимости, а редукция выполняется в том числе под λ и Λ.

Применение тождественной функции

$$(\Lambda\alpha.\,\lambda x : \alpha.\,x)[\mathsf{Int}]\,42$$

$$\to_{\beta_\forall} \quad (\lambda x : \mathsf{Int}.\,x)\,42$$

После подстановки Int параметр x получает аннотацию Int.

$$\to_{\beta} \quad 42$$

Затем вместо x подставляем 42.

На всех шагах выражение имеет тип Int.

Выбор первого и второго аргумента

$\mathsf{first} \coloneqq \Lambda\alpha.\,\Lambda\beta.\,\lambda x : \alpha.\,\lambda y : \beta.\,x$

$\mathsf{first} : \forall\alpha.\,\forall\beta.\,\alpha \to \beta \to \alpha$

$\mathsf{second} \coloneqq \Lambda\alpha.\,\Lambda\beta.\,\lambda x : \alpha.\,\lambda y : \beta.\,y$

$\mathsf{second} : \forall\alpha.\,\forall\beta.\,\alpha \to \beta \to \beta$

$\mathsf{second}[\mathsf{Bool}][\mathsf{Int}]\ \mathsf{true}\ 42 \to^*_{\beta,\beta_\forall} 42$

Типы Bool и Int подставляются в порядке абстракций $\Lambda\alpha$ и $\Lambda\beta$.

Самоприменение в System F

$$\mathsf{I} \coloneqq \forall\alpha.\,\alpha \to \alpha \qquad \Gamma = f : \mathsf{I}$$

$$\Gamma \vdash f[\mathsf{I}] : \mathsf{I} \to \mathsf{I} \qquad \Gamma \vdash f : \mathsf{I}$$

$$\frac{\Gamma \vdash f[\mathsf{I}] : \mathsf{I} \to \mathsf{I} \qquad \Gamma \vdash f : \mathsf{I}}{\Gamma \vdash f[\mathsf{I}]\, f : \mathsf{I}}$$

Для левого вхождения f выбираем типовой аргумент I, получая $f[\mathsf{I}] : \mathsf{I} \to \mathsf{I}$. Правое вхождение f имеет тип I.

Самоприменение и общая рекурсия

$\lambda f \,:\, \mathsf{I}.\, f\,[\mathsf{I}]\, f \,:\, \mathsf{I} \to \mathsf{I}$

Стирание типов даёт $\lambda f.\, f\ f$, поэтому этот терм **типизируется в стиле Карри**.

$\mathsf{id}[\mathsf{I}]\,\mathsf{id} \to_{\beta_\forall} (\lambda x \,:\, \mathsf{I}.\, x)\,\mathsf{id} \to_{\beta} \mathsf{id}$

В примере с id самоприменение за два шага возвращает id.

Все типизируемые термы чистой System F сильно нормализуются, поэтому для общей рекурсии исчисление приходится расширять.

Два мономорфных инстанцирования

$\Gamma = f : \mathsf{I}, \beta : *$

$f[\beta \to \beta] : (\beta \to \beta) \to (\beta \to \beta)$

$f[\beta] : \beta \to \beta$

$f[\beta \to \beta]\,(f[\beta]) : \beta \to \beta$

$\lambda f : \mathsf{I}.\, \Lambda\beta.\, f[\beta \to \beta]\,(f[\beta]) : \mathsf{I} \to \mathsf{I}$

Оба типовых аргумента, β и $\beta \to \beta$, мономорфны, а стирание снова даёт $\lambda f.\, f\ f$.

Три ошибки типизации

Пустой контекст

1. $\Lambda\alpha.\,\alpha$
2. $\Lambda\alpha.\,\lambda x : \beta.\,x$
3. $(\Lambda\alpha.\,\lambda x : \alpha.\,x)\,42$

1. Тело типовой абстракции должно быть термом.

2. Для аннотации $x : \beta$ требуется объявление $\beta : *$.

3. Перед аргументом 42 требуется типовая аппликация [Int].

Метатеория System F

Сохранение типов

$$\Gamma \vdash M : \tau \quad \text{и} \quad M \to_{\beta,\beta_\forall} N \quad \Longrightarrow \quad \Gamma \vdash N : \tau$$

Сильная нормализация

Любая последовательность термовых и типовых β-редукций типизируемого терма конечна.

Конфлюэнтность

Любые два результата редукции данного терма имеют общий редукт.

План лекции

1 Квантор ∀ и правила System F

2 Стиль Карри и вывод типов Hindley–Milner

3 Параметричность и свойства программ

System F в стиле Карри

$$m, n ::= x \mid \lambda x.\, m \mid m\, n$$

$$\frac{\Gamma, \alpha : * \vdash m : \tau}{\Gamma \vdash m : \forall \alpha.\, \tau} \qquad \frac{\Gamma \vdash m : \forall \alpha.\, \tau \quad \Gamma \vdash \sigma : *}{\Gamma \vdash m : \tau[\alpha := \sigma]}$$

Термы состоят из переменных, абстракций и аппликаций, а типы присваиваются правилами вывода.

Стирание аннотаций

$$|\lambda x : \tau. M| = \lambda x. |M| \qquad |M\,N| = |M|\,|N|$$

$$|\Lambda\alpha. M| = |M| \qquad |M[\tau]| = |M| \qquad |x| = x$$

$$|\Lambda\alpha. \lambda x : \alpha. x| = \lambda x. x$$

$$|\lambda f : \mathsf{I}. f[\mathsf{I}]\, f| = \lambda f. f\ f$$

Стирание удаляет аннотации параметров, типовые абстракции и типовые аппликации.

Задачи проверки, вывода и обитаемости

Проверка типа

Для заданных Γ, M и τ проверить суждение $\Gamma \vdash M : \tau$.

Вывод типа

Для заданных Γ и M определить типизируемость и при положительном ответе найти тип M.

Обитаемость типа

Для заданного τ определить, существует ли замкнутый терм M типа τ.

Разрешимость в полной System F

Задача	Чёрч: явные типы	Карри: стёртый терм
Проверка типа	Разрешима	Неразрешима
Вывод типа	Разрешима	Неразрешима
Обитаемость типа	Неразрешима	Неразрешима

У полностью аннотированного терма тип определяется по синтаксису с точностью до α-переименования.

Для стиля Карри проверка типа и типизируемость полной System F алгоритмически неразрешимы.

Проверка типа явного терма

$$\mathsf{type}_{\Gamma}(\Lambda\alpha.\, M) = \forall\alpha.\, \mathsf{type}_{\Gamma,\alpha:*}(M)$$

При проверке типовой аппликации $M[\sigma]$ сначала находим тип M.

1. Проверяем, что найденный тип имеет вид $\forall\alpha.\, \tau$.
2. Проверяем корректность σ в текущем контексте.
3. Возвращаем $\tau[\alpha := \sigma]$.

В обычной аппликации тип аргумента должен совпадать с областью определения функции **с точностью до α-переименования**.

Hindley–Milner: мономорфные типы и схемы

$$\tau ::= \alpha \mid \tau \to \tau$$

$$\sigma ::= \forall \alpha_1 \ldots \alpha_n . \tau$$

Мономорфный тип τ строится из переменных и стрелок, а **схема** σ связывает его переменные кванторами снаружи.

Параметры λ получают мономорфные типы. При связывании в `let` тип выражения обобщается в схему.

При каждом обращении к имени связанные переменные его схемы заменяются **свежими типовыми переменными**.

Полиморфизм let и параметр функции

Полиморфное связывание в let

```
let pair =
  let id x = x in
  (id 42, id true)
```

Два обращения к `id` независимо инстанцируют схему $\forall\alpha.\,\alpha \to \alpha$.

В чистом HM параметр f мономорфен

```
let use f = (f 42, f true)
```

У f один мономорфный тип, а вызовы требуют одновременно $f : \mathrm{Int} \to \gamma$ и $f : \mathrm{Bool} \to \delta$.

Какие переменные можно обобщить

$$\mathrm{Gen}(\Gamma, \tau) = \forall \overline{\alpha}.\, \tau$$

$$\{\overline{\alpha}\} = \mathrm{FTV}(\tau) \setminus \mathrm{FTV}(\Gamma)$$

Свободные переменные, общие для τ и Γ, связывают тип выражения с внешними значениями.

$$\lambda x.\, \mathsf{let}\ f = \lambda y.\, x\ \mathsf{in}\ f$$

$$x : \alpha \quad \Longrightarrow \quad f : \forall \beta.\, \beta \to \alpha$$

При каждом вызове f тип β выбирается заново, а тип результата α определяется внешним x.

Восстановление явных типов

Исходная программа

$\mathsf{let}\ id = \lambda x.\, x\ \mathsf{in}\ (id\, 42, id\, \mathsf{true})$

После восстановления типов

$\mathsf{let}\ id = \Lambda\alpha.\, \lambda x : \alpha.\, x$

$\mathsf{in}\ (id[\mathsf{Int}]\, 42, id[\mathsf{Bool}]\, \mathsf{true})$

При обобщении вводим Λ-абстракцию, а при обращении к имени подставляем выбранный тип.

В явной записи $\mathsf{let}\ x = M\ \mathsf{in}\ N$ означает $(\lambda x : \sigma.\, N)M$, где $M : \sigma$.

Полиморфный аргумент функции

$$\mathsf{use} : (\forall \alpha.\, \alpha \to \alpha) \to \mathsf{Int} \times \mathsf{Bool}$$

$$\mathsf{use} \coloneqq \lambda f : \mathsf{I}.\, (f[\mathsf{Int}]\, 42, f[\mathsf{Bool}]\, \mathsf{true})$$

Функция use применяет f к числу и булеву значению, поэтому принимает **полиморфную функцию**.

System F разрешает такую аннотацию параметра. В чистом HM параметры функций имеют мономорфные типы.

Тип use имеет **ранг 2**, поскольку квантор находится внутри типа аргумента. Вызовы f используют мономорфные типовые аргументы.

Ранг и импредикативность

Ранг: где расположен квантор

$$(\forall \alpha.\, \alpha \to \alpha) \to \mathsf{Int}$$

Этот тип имеет ранг 2: аргументом служит функция, которую можно инстанцировать разными типами.

Импредикативность: чем инстанцируется переменная

$$\mathsf{id}[\forall \alpha.\, \alpha \to \alpha] \;:\; \mathsf{I} \to \mathsf{I}$$

При импредикативном инстанцировании вместо переменной подставляется полиморфный тип.

Катаморфизм: свёртка дерева

```
type expr = Lit of int | Add of expr * expr

let rec fold_expr lit add = function
  | Lit n -> lit n
  | Add (left, right) ->
      add (fold_expr lit add left)
          (fold_expr lit add right)
```

$F(X) = \mathsf{Int} + X \times X \qquad \alpha : F(A) \to A$

В узле `Lit` вызывается `lit`, а в узле `Add` функция `add` объединяет результаты свёртки поддеревьев.

Операции `lit` и `add` задают **алгебру** $\alpha : F(A) \to A$ с носителем A.

Вычисление значения и размера дерева

```
let example = Add (Lit 2, Add (Lit 3, Lit 4))
```

Вычисление значения

```
let eval = fold_expr (fun n -> n) ( + )
```

$$\text{eval example} = 2 + (3 + 4) = 9$$

Подсчёт всех узлов

```
let size =
  fold_expr (fun _ -> 1) (fun l r -> 1 + l + r)
```

$$\text{size example} = 1 + 1 + (1 + 1 + 1) = 5$$

Вывод типов с окружением

Тип вхождения x в теле $\lambda x.\, x$ определяется объявлением параметра.

Сворачивая синтаксическое дерево, для каждого подтерма строим **функцию от окружения** и запаса свежих переменных.

$$A = \mathsf{Env} \to \mathsf{Fresh} \to \mathsf{Result}$$

$$\mathsf{Result} = \mathsf{Error} + (\mathsf{Subst} \times \mathsf{MonoType} \times \mathsf{Fresh})$$

Для $\lambda x.\, M$ выбираем свежую α и вызываем анализатор тела в окружении $\Gamma, x : \alpha$.

Аппликация в алгоритме W

Для подтермов M и N построены анализаторы K_M и K_N. При обработке аппликации вызываем их последовательно.

$$(S_1, \tau_1) = K_M(\Gamma)$$

$$(S_2, \tau_2) = K_N(S_1\Gamma)$$

Перед анализом N применяем к окружению подстановку S_1.

$$U = \mathrm{mgu}(S_2\tau_1, \tau_2 \to \beta) \qquad \beta \text{ свежая}$$

$$S = U \circ S_2 \circ S_1 \qquad \tau = U\beta$$

mgu находит наиболее общий унификатор. Ошибка любого шага становится результатом анализа всей аппликации.

Главная схема типа в Hindley–Milner

Для типизируемого выражения HM алгоритм W находит **главную схему типа**, инстанцирования которой дают все его допустимые типы.

$$\lambda f.\,\lambda x.\,f\ x\ :\ \forall\alpha.\,\forall\beta.\,(\alpha \to \beta) \to \alpha \to \beta$$

При обходе терма W создаёт свежие переменные, унифицирует типы и выполняет обобщение в `let`.

Окружение и подстановка передаются между шагами, как в разобранном случае аппликации.

Задача типизируемости чистого HM является EXPTIME-полной.

План лекции

1 Квантор ∀ и правила System F

2 Стиль Карри и вывод типов Hindley–Milner

3 Параметричность и свойства программ

Термы типа тождественной функции

$$f \;:\; \forall \alpha.\, \alpha \to \alpha$$

В контексте $x : \alpha$ единственная нормальная форма типа α — сам аргумент x.

$$f[A]\, x \;=_{\beta\eta}\; x$$

В чистой System F любой **замкнутый** терм этого типа βη-эквивалентен id.

Параметричность через отношения

Замкнутая функция $f : \forall\alpha.\, \alpha \to \alpha$ сохраняет любое отношение $R \subseteq A \times B$ при инстанцировании типами A и B.

$$(a, b) \in R \quad \Longrightarrow \quad (f[A]\, a, f[B]\, b) \in R$$

Для списков

ListRel(R) связывает списки одинаковой длины с попарно связанными элементами.

$$([], []) \in \text{ListRel}(R) \qquad (a :: xs, b :: ys) \in \text{ListRel}(R)$$

Для второй пары нужны условия $(a, b) \in R$ и $(xs, ys) \in \text{ListRel}(R)$.

Теорема из типа для списков

$$r : \forall\alpha.\, \mathsf{List}\,\alpha \to \mathsf{List}\,\alpha$$

Пусть r — замкнутая полиморфная функция, $h : A \to B$ тотальна, а xs — конечный список.

$$r[B]\,(\mathsf{map}\,h\,xs) = \mathsf{map}\,h\,(r[A]\,xs)$$

Равенство выполняется для **любой замкнутой реализации** r этого типа в чистой тотальной модели.

Например, для r = reverse применение h к элементам можно выполнить до или после обращения списка.

Доказательство теоремы для списков

1. Выбираем граф функции $h : A \to B$

$$R_h = \{(a, h(a)) \mid a \in A\}$$

2. Связываем входные списки

$$(xs, \mathsf{map}\ h\ xs) \in \mathsf{ListRel}(R_h)$$

3. Применяем параметричность r

$$(r[A]\ xs, r[B]\ (\mathsf{map}\ h\ xs)) \in \mathsf{ListRel}(R_h)$$

4. Раскрываем смысл $\mathsf{ListRel}(R_h)$

$$r[B]\ (\mathsf{map}\ h\ xs) = \mathsf{map}\ h\ (r[A]\ xs)$$

Полиморфные функции и длина списка

$g : \forall\alpha.\,\forall\beta.\,(\alpha \to \beta) \to \mathrm{List}\,\alpha \to \mathrm{List}\,\beta$

Сохраняет ли такая функция длину списка?

```
let drop_all f xs = []
```

```
let duplicate f xs =
  List.concat_map (fun x -> f x :: f x :: []) xs
```

Функция `drop_all` возвращает пустой список, а `duplicate` удваивает длину. Обе функции можно использовать с указанной схемой типа.

Первый элемент и пустой список

$\mathsf{head} : \forall\alpha.\, \mathsf{List}\,\alpha \to \alpha$

Для пустого типа $\mathbf{o}$ существует пустой список $[] : \mathsf{List}\,\mathbf{o}$.

$[] : \mathsf{List}\,\mathbf{o} \qquad \mathsf{head}[\mathbf{o}]\,[] : \mathbf{o}$

Применение head к этому списку дало бы замкнутый терм пустого типа.

$\mathsf{headOption} : \forall\alpha.\, \mathsf{List}\,\alpha \to \mathsf{Option}\,\alpha$

Можно также принимать список с первым элементом, представленный парой $\alpha \times \mathsf{List}\,\alpha$.

Первый элемент с результатом Option

```
let head_option = function
  | [] -> None
  | x :: _ -> Some x
```

$$\text{headOption}[B]\,(\text{map}\,h\,xs) =$$

$$\quad \text{mapOption}\,h\,(\text{headOption}[A]\,xs)$$

Для $[]$ обе стороны равны None, а для $x :: xs$ возвращают $\text{Some}\,(h\,x)$.

Условия параметричности

Чистая System F

Предыдущие теоремы относятся к чистой System F с сильной нормализацией.

Общая рекурсия

В языке с общей рекурсией функция типа $\forall \alpha. \alpha \to \alpha$ может зациклиться.

Сведения о типах

В Haskell ограничение `Typeable a` даёт доступ к представлению типа `a` во время выполнения.

Вопросы

1. Типизируется ли $\lambda f : \mathsf{I}.\, f\,[\mathsf{I}]\, f$?
2. Гарантирует ли $\forall \alpha.\, \mathsf{List}\,\alpha \to \mathsf{List}\,\alpha$ сохранение длины?
3. Разрешима ли проверка полностью аннотированного терма System F?

1. Да, тип $\mathsf{I} \to \mathsf{I}$.

2. Можно вернуть пустой список при любом входе.

3. Да, тип определяется по синтаксису терма.

Итоги

В **System F** типовые параметры вводятся абстракцией Λ и подставляются типовой аппликацией.

Для выражений **Hindley–Milner** алгоритм W автоматически находит главные схемы типов.

Параметричность связывает поведение разных типовых инстанцирований одной полиморфной функции.

Дополнительные темы

Кодировки и экзистенциальные типы

Булевы значения, пары, списки и натуральные числа как полиморфные функции.

Скрытое представление данных через экзистенциальный тип.

Ограничение обобщения в языках с эффектами.

Булевы значения как выбор

$$\mathsf{Bool}_C := \forall\alpha.\, \alpha \to \alpha \to \alpha$$

$$\mathsf{true}_C := \Lambda\alpha.\, \lambda x : \alpha.\, \lambda y : \alpha.\, x$$

$$\mathsf{false}_C := \Lambda\alpha.\, \lambda x : \alpha.\, \lambda y : \alpha.\, y$$

Булеву значению передаём тип результата и два аргумента, из которых оно выбирает один.

$$\mathsf{true}_C\,[\mathsf{Int}]\;42\;0 \to^*_{\beta,\beta_\forall} 42$$

В чистой System F этот тип имеет ровно два замкнутых терма с точностью до βη-эквивалентности.

Кодировка пары

$$\mathsf{Pair}\, A\, B \coloneqq \forall \rho.\, (A \to B \to \rho) \to \rho$$

Пара передаёт компоненты функции $k : A \to B \to \rho$, которая возвращает результат типа ρ.

$$\mathsf{pair}[A][B]\, a\, b \coloneqq \Lambda \rho.\, \lambda k : A \to B \to \rho.\, k\, a\, b$$

$$\mathsf{fst}[A][B]\, p \coloneqq p[A]\, (\lambda a : A.\, \lambda b : B.\, a)$$

$$\mathsf{snd}[A][B]\, p \coloneqq p[B]\, (\lambda a : A.\, \lambda b : B.\, b)$$

Переменная ρ выбирается свежей для A, B и контекста.

Вычисление первой проекции

$\mathsf{fst}[A][B]\,(\mathsf{pair}[A][B]\,a\,b)$

$\to^*_{\beta,\beta_\mathsf{v}}\ (\Lambda\rho.\,\lambda k : A \to B \to \rho.\,k\,a\,b)[A]\,(\lambda x : A.\,\lambda y : B.\,x)$

$\to^*_{\beta,\beta_\mathsf{v}}\ (\lambda x : A.\,\lambda y : B.\,x)\,a\,b \to^*_{\beta}\ a$

Вторая проекция аналогично возвращает b.

В проекции fst выбираем $\rho = A$, а в snd — $\rho = B$.

Список как его свёртка

$$\mathsf{List}_C\, A \coloneqq \forall \rho.\, (A \to \rho \to \rho) \to \rho \to \rho$$

Чтобы свернуть список xs, передаём тип результата ρ, шаг c и начальное значение z.

$$xs : \mathsf{List}_C\, A \quad \Longrightarrow \quad xs[\rho]\, c\, z : \rho$$

$$\mathsf{foldr}\, c\, z\, xs \coloneqq xs[\rho]\, c\, z$$

Значение z соответствует пустому списку, а функция c обрабатывает очередной элемент и результат свёртки хвоста.

Пустой список и добавление элемента

Пустой список возвращает начальное значение

$$\mathsf{nil}[A] \coloneqq \Lambda\rho.\,\lambda c : A \to \rho \to \rho.\,\lambda z : \rho.\,z$$

Конструктор cons применяет один шаг свёртки

$$\mathsf{cons}[A]\;a\;xs \coloneqq$$

$$\Lambda\rho.\,\lambda c : A \to \rho \to \rho.\,\lambda z : \rho.\,c\;a\;(xs[\rho]\;c\;z)$$

$$\mathsf{cons} : \forall\alpha.\,\alpha \to \mathsf{List}_C\;\alpha \to \mathsf{List}_C\;\alpha$$

При свёртке $\mathsf{cons}[A]\;a\;xs$ функция c получает первый элемент a и результат свёртки хвоста.

Преобразование списка через свёртку

$$\mathsf{map}_C[A][B]\,h\,xs \coloneqq$$
$$\Lambda\rho.\,\lambda c : B \to \rho \to \rho.\,\lambda z : \rho.$$
$$xs[\rho]\,(\lambda a : A.\,\lambda r : \rho.\,c\,(h\,a)\,r)\,z$$

$h : A \to B \qquad xs : \mathsf{List}_C\,A$

$\mathsf{map}_C[A][B]\,h\,xs : \mathsf{List}_C\,B$

В шаге свёртки к элементу a применяется h, а значение $h\,a$ передаётся функции c.

Натуральное число как итератор

$\mathsf{Nat}_C \coloneqq \forall\alpha.\,(\alpha \to \alpha) \to \alpha \to \alpha$

$\overline{0} \coloneqq \Lambda\alpha.\,\lambda s : \alpha \to \alpha.\,\lambda z : \alpha.\,z$

$\overline{2} \coloneqq \Lambda\alpha.\,\lambda s : \alpha \to \alpha.\,\lambda z : \alpha.\,s\,(s\,z)$

$\mathsf{succ}_C\ n \coloneqq \Lambda\alpha.\,\lambda s : \alpha \to \alpha.\,\lambda z : \alpha.\,s\,(n[\alpha]\,s\,z)$

Число $\overline{n}$ применяет функцию s к значению z ровно n раз.

$\overline{2}[\mathsf{Int}]\,(\lambda n : \mathsf{Int}.\,n+1)\,0 \to^*_{\beta,\beta_v} (0+1)+1 = 2$

Скрытое представление счётчика

$$\mathsf{Counter} := \exists \alpha.\, \alpha \times (\alpha \to \alpha) \times (\alpha \to \mathsf{Int})$$

В пакет счётчика входят тип состояния α, начальное значение zero и операции inc и get.

$$\mathsf{zero} : \alpha \qquad \mathsf{inc} : \alpha \to \alpha \qquad \mathsf{get} : \alpha \to \mathsf{Int}$$

$$\mathsf{get}\,(\mathsf{inc}\,\mathsf{zero}) : \mathsf{Int}$$

От реализации счётчика требуем $\mathsf{get}\,(\mathsf{inc}\,s) = \mathsf{get}\,s + 1$.

Экзистенциальный тип через ∀

$$\exists \alpha.\, \tau \quad := \quad \forall \rho.\, (\forall \alpha.\, \tau \to \rho) \to \rho$$

Для распаковки передаём функцию k, принимающую содержимое пакета **при любом типе представления** α.

$$k : \forall \alpha.\, \tau \to \rho$$

Тип результата ρ выбирается до распаковки и остаётся одним и тем же для всех представлений α.

Переменная ρ свежая: $\rho \notin \mathrm{FTV}(\tau)$. В типе k квантор связывает α во всём $\tau \to \rho$.

Упаковка конкретного представления

$$\Gamma \vdash \sigma : * \qquad \Gamma \vdash M : \tau[\alpha := \sigma]$$

$$\mathsf{pack}[\sigma, M] \coloneqq \Lambda\rho.\, \lambda k : (\forall\alpha.\, \tau \to \rho).\, k[\sigma]\, M$$

$$\Gamma \vdash \mathsf{pack}[\sigma, M] : \exists\alpha.\, \tau$$

Операция pack скрывает выбранный тип σ за квантором $\exists\alpha$.

Для счётчика можно выбрать $\sigma = \mathsf{Int}$, начальное значение 0 и функции $\lambda n : \mathsf{Int}.\, n + 1$ и $\lambda n : \mathsf{Int}.\, n$.

Распаковка и область скрытого типа

$$\Gamma \vdash E : \exists \alpha.\, \tau \qquad \Gamma, \alpha : *, x : \tau \vdash N : R$$

$$\Gamma \vdash R : * \qquad \alpha \notin \mathrm{FTV}(R)$$

$$\mathsf{unpack}\ (\alpha, x) = E\ \mathsf{in}\ N \coloneqq E[R]\,(\Lambda \alpha.\, \lambda x : \tau.\, N)$$

Возвращаемый тип R задаётся во **внешнем контексте** Γ.

После распаковки счётчика выражение get (inc zero) имеет внешний тип Int.

Значение скрытого типа можно вернуть в новом экзистенциальном пакете.

Полиморфизм и изменяемые ссылки

```
let bad =
  let r = ref (fun x -> x) in
  let () = r := (fun n -> n + 1) in
  (!r) true
```

При обобщении типа `r` одна ссылка допускала бы запись функции Int → Int и её чтение с типом Bool → Bool.

В OCaml после присваивания `r` имеет тип `(int -> int) ref`, поэтому вызов `(!r) true` даёт ошибку типизации.

Обобщение в присутствии эффектов

Классическое **ограничение на обобщение** (value restriction) разрешает полиморфизм `let`, когда справа стоит значение, например λ-абстракция.

```
let id x = x
```

```
let id = (fun f -> f) (fun x -> x)
```

Во втором определении `id` имеет слабую переменную типа, которая фиксируется при первом использовании.

В OCaml **ослабленное ограничение** также разрешает обобщать ковариантные переменные в типах вычисляемых выражений.

η-законы и эквивалентность кодировок

η-равенства функций в чистой System F

$$\lambda x : \sigma.\, M\, x \;=_\eta\; M \qquad x \notin \mathrm{FV}(M)$$

$$\Lambda\alpha.\, M[\alpha] \;=_\eta\; M \qquad \alpha \notin \mathrm{FTV}(M)$$

Восстановление пары из проекций

$$\mathsf{pair}[A][B]\,(\mathsf{fst}[A][B]\;p)\,(\mathsf{snd}[A][B]\;p) \;\simeq\; p$$

Для кодировки Чёрча это равенство следует из параметричности.

$\simeq$ — наблюдаемая эквивалентность в чистой тотальной модели.

Упражнения

1. Запишите явный терм типа $\forall\alpha.\,\forall\beta.\,\alpha \to \beta \to \beta$.
2. Найдите тип $\mathsf{id}[\mathsf{I}]$ и результат $\mathsf{id}[\mathsf{I}]\,\mathsf{id}$.
3. Приведите функцию типа $\forall\alpha.\,\mathsf{List}\,\alpha \to \mathsf{List}\,\alpha$, меняющую порядок элементов.
4. Дополните пакет $\exists\alpha.\,\alpha$ операцией чтения значения типа Int.

Обоснуйте каждый ответ.

Ответы к упражнениям

1. Терм

$\Lambda\alpha.\,\Lambda\beta.\,\lambda x : \alpha.\,\lambda y : \beta.\,y$

2. Тип и редукция

$\mathsf{id}[\mathsf{I}] : \mathsf{I} \to \mathsf{I} \qquad \mathsf{id}[\mathsf{I}]\,\mathsf{id} \to^{*}_{\beta,\beta_\forall} \mathsf{id}$

3. Обращение списка

Функция reverse меняет порядок элементов и удовлетворяет закону с map.

4. Операция чтения

Пакет $\exists\alpha.\,\alpha \times (\alpha \to \mathsf{Int})$ содержит значение и функцию, возвращающую результат типа Int.

Материалы к лекции

System F и правила вычисления

P. Selinger, Lecture Notes on the Lambda Calculus, раздел 8

Вывод типов и параметричность

L. Damas, R. Milner, Principal Type-Schemes for Functional Programs

A. Dudenhefner, System F Typability and Type Checking

P. Wadler, Parametricity и Theorems for Free

Примеры OCaml

`lecture_5_examples.ml`: полиморфизм `let`, свёртки выражений и преобразования списков.

OCaml Manual: Polymorphism and its limitations